



資料結構

Data Structure

Lab 11

姓名：夏宇柔
學號：113840005

Lab11-Ex1.

Please write a function that receives the layer number and returns the sum of all nodes at that layer. Please also discuss various defensive programming mechanisms.

Code

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;
const int EMPTY = NULL; // 用 NULL 代表沒有節點
class TreeNode {
public:
    int value;          // 節點值
    TreeNode* left;     // 左子節點
    TreeNode* right;    // 右子節點

    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {} // 初始化節點
};

class BinaryTree {
public:
    TreeNode* root; // 樹根節點

    BinaryTree() : root(nullptr) {}

    TreeNode* buildTree(const vector<int>& arr) { // 建立樹
        if (arr.empty() || arr[0] == EMPTY) return nullptr;

        queue<TreeNode*> q; // 建立queue儲存待處理的節點指標
        root = new TreeNode(arr[0]); // 建立根節點 (陣列第一個元素)
        q.push(&root); // 將根節點的指標加入queue

        size_t i = 1; // 陣列索引
        while (!q.empty() && i < arr.size()) {
            TreeNode** nodePtr = q.front(); // 取出queue的第一個節點指標
            q.pop(); // 將該節點從queue中刪除

            // 左子節點
            if (i < arr.size()) {
                if (arr[i] != EMPTY) {
                    (*nodePtr)->left = new TreeNode(arr[i]); // 添加左子節點
                    q.push(&((*nodePtr)->left)); // 將左子節點的指標加入queue
                }
                i++;
            }

            // 右子節點
            if (i < arr.size()) {
                if (arr[i] != EMPTY) {
                    (*nodePtr)->right = new TreeNode(arr[i]); // 添加右子節點
                    q.push(&((*nodePtr)->right)); // 將右子節點的指標加入queue
                }
            }
        }
    }
};
```

```

        i++;
    }
}
return root;
}

void Depth_first_search(TreeNode* node) { //深度優先搜尋
    if (node == nullptr) return;
    cout << node->value << " "; // 訪問當前節點的值
    Depth_first_search(node->left); // 遞迴遍歷左子樹
    Depth_first_search(node->right); // 遞迴遍歷右子樹
}

void Breadth_first_search(TreeNode* root) {
    if (root == nullptr) return;
    queue<TreeNode*> q; // 建立queue儲存待處理的節點指標
    q.push(root); // 將根節點的指標加入queue

    while (!q.empty()) {
        TreeNode* current = q.front(); // 取出queue的第一個節點指標
        q.pop(); // 將該節點從queue中刪除
        cout << current->value << " ";
        if (current->left) q.push(current->left); // 將左子節點的指標加入
queue
        if (current->right) q.push(current->right); // 將右子節點的指標加入
queue
    }
}

void levelSum(int level, TreeNode* root) {
    if (root == nullptr) { // if root is null, print tree is empty.
        cout << "Tree is empty.";
        return;
    }
    queue<TreeNode*> q; // create queue to save nodes for processing
    q.push(root); // push root into queue
    int currentLevel = 0; // current level is 0, the root
    int sum = 0; // initialize sum
    bool flag = false; // initialize flag; determines if a sum of the
requested level was calculated- if not, input layer is invalid

    while (!q.empty()) { // run until no more nodes
        int levelSize = q.size(); // find the size of the current
level(all nodes already put into queue)
        for (int i = 0; i < levelSize; i++) { // loop through all nodes of
current level
            TreeNode* current = q.front(); // retrieve first node from
queue
            q.pop(); // remove first node from
queue

            if (currentLevel == level) { // if this is the target level
                sum += current->value; // add this node's value to the
sum
                flag = true; // change flag to true
            }
        }
    }
}

```

```

        if (current->left) q.push(current->left);    // add left node
into queue
        if (current->right) q.push(current->right); // add right node
into queue
    }
    if (flag) { // if sum was calculated
        break; // we can leave loop early
    }
    currentLevel++; // move on to next level
}
if (flag) { // if sum was calculated, output result
    cout << "Sum: " << sum << endl;
    return;
}
// otherwise assume input layer was invalid
cout << "Input layer invalid." << endl;
return;
}
};

int main() {
    BinaryTree tree;
    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7, 8, 9, NULL, NULL, 10, 11, NULL,
NULL };
    tree.buildTree(arr);

    cout << "DFS Result: ";
    tree.Depth_first_search(tree.root);
    cout << endl;

    cout << "BFS Result: ";
    tree.Breadth_first_search(tree.root);
    cout << endl;

    int lvl = 0; // initialize input integer
    cout << "Please enter the layer to query, starting from 0: "; // request
user input
    cin >> lvl; // user input
    tree.levelSum(lvl, tree.root); // begin search

    system("pause");
    return 0;
}

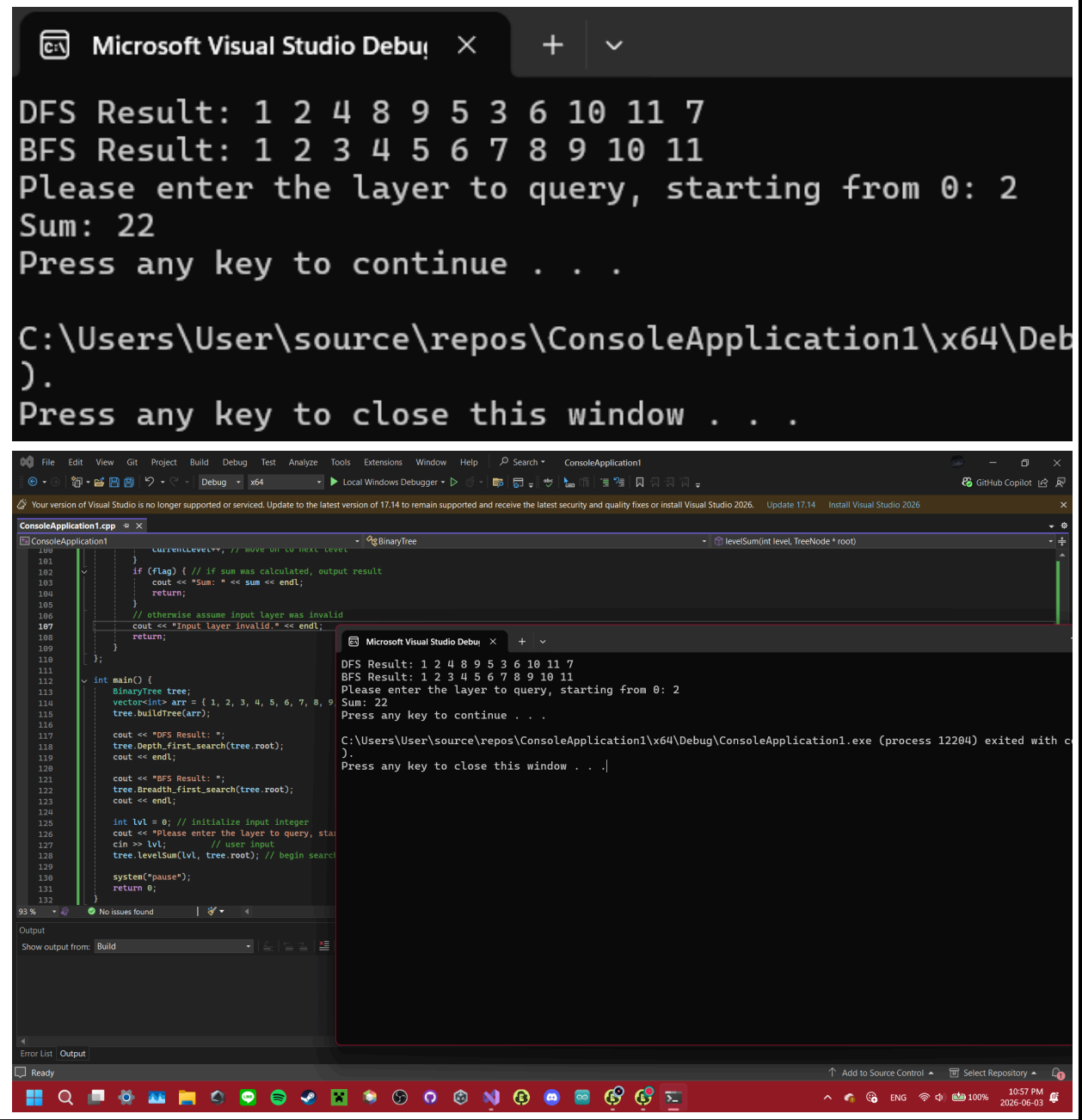
```

Discussion Section

Defensive programming mechanisms I implemented:

1. In the beginning of the function, I included a guard clause for if the tree is null- if so, outputs "Tree is empty" and exits the function.
2. Uses a flag to determine if the sum was ever calculated- in turn, this ensures that any input value that cannot produce a valid sum will be flagged and produce an "Input layer invalid" output.
3. With the same flag, I added an early exit for the while loop: if the layer has already been calculated, we no longer need to look at the rest of the tree, and can exit early, saving computation time.

Screenshots:





Microsoft Visual Studio Debug



DFS Result: 1 2 4 8 9 5 3 6 10 11 7

BFS Result: 1 2 3 4 5 6 7 8 9 10 11

Please enter the layer to query, starting from 0: 4

Input layer invalid.

Press any key to continue . . .

C:\Users\User\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe (process 3196) exited with code 0.

Press any key to close this window . . .

